

Práctica n°2. Introducción a los Sistemas Operativos

1. Editor de textos :

A. Nombre al menos 3 editores de texto que puede utilizar desde la línea de comandos.

- Vim : Es una versión avanzada del editor **vi**. Ofrece características adicionales como resaltado de sintaxis, autocompletado y múltiples niveles de deshacer.
- Nano : Es un editor de texto de línea de comandos fácil de usar. Ofrece una interfaz intuitiva con atajos de teclado visibles en la parte inferior de la pantalla.
- Mcedit : es el editor de texto incluido en **Midnight Commander (MC)**, que es una herramienta de administración de archivos de texto basada en la interfaz de línea de comandos para sistemas Unix/Linux. Proporciona una interfaz de usuario visual de tipo panel, lo que facilita la navegación y la gestión de archivos, así como la edición de archivos de texto.

B. ¿En qué se diferencia un editor de texto de los comandos cat, more o less?
Enumere los modos de operación que posee el editor de textos vi.

Los comandos cat , more y less son comandos que están diseñados para visualizar el contenido de archivos, no para crear o modificar archivos como un editor de texto.

Posee tres modos de operación :

- Modo de comando.(Podemos ejecutar comandos para manipular el texto)
- Modo de inserción. (Insertar o modificar texto)
- Modo línea de comando.(Comandos para guardar, salir, buscar, y realizar otras tareas)

C. Nombre los comandos más comunes que se le pueden enviar al editor de textos vi .

COMANDOS PERO EN LOS 3 MODOS O EN ALGUNO EN ESPECIAL ?

2. Proceso de Arranque SystemV:

A. Enumere los pasos del proceso de inicio de un sistema GNU/Linux, desde que se prende la PC hasta que se logra obtener el login en el sistema.

Proceso de arranque System V

(<https://github.com/systeminit/si>)

- ❶ Se empieza a ejecutar el código del BIOS
- ❷ El BIOS ejecuta el POST
- ❸ El BIOS lee el sector de arranque (MBR)
- ❹ Se carga el gestor de arranque (MBC)
- ❺ El bootloader carga el *kernel* y el *initrd*
- ❻ Se monta el *initrd* como sistema de archivos raíz y se inicializan componentes esenciales (ej.: scheduler)
- ❼ El Kernel ejecuta el proceso *init* y se desmonta el *initrd*
- ❽ Se lee el */etc/inittab*
- ❾ Se ejecutan los scripts apuntados por el *runlevel 1*
- ❿ El final del *runlevel 1* le indica que vaya al *runlevel* por defecto
- ⓫ Se ejecutan los scripts apuntados por el *runlevel* por defecto
- ⓬ El sistema está listo para usarse



Facultad de Informática
UNIVERSIDAD NACIONAL DE LA PLATA

B. Proceso INIT. ¿Quién lo ejecuta? ¿Cuál es su objetivo?.

Es ejecutado por el Kernel.

- - Su función es cargar todos los subprocesos necesarios para el correcto funcionamiento del SO.
- - El proceso *init* posee el PID 1 y se encuentra en */sbin/init*.
- - En SysV se lo configura a través del archivo */etc/inittab*.
- - No tiene padre y es el padre de todos los procesos (*ps tree*).
- - Es el encargado de montar los filesystems y de hacer disponible los demás dispositivos.

C. RunLevels. ¿Qué son? ¿Cuál es su objetivo?.

Es el modo en que arranca Linux (3 en Redhat, 2 en Debian).

El proceso de arranque se divide en 3 niveles, cada uno responsable de levantar (Iniciar) o bajar (Parar) una serie de archivos.

(Un nivel de ejecución es una configuración de programas y servicios que se ejecutarán orientados a un determinado funcionamiento).

D. ¿A qué hace referencia cada nivel de ejecución según el estándar?

- Existen 7, y permiten iniciar un conjunto de procesos al arranque o apagado del sistema
- Según el estándar:
 - **0**: halt (parada)
 - **1**: single user mode (monousuario)
 - **2**: multiuser, without *NFS* (modo multiusuario sin soporte de red)
 - **3**: full multiuser mode console (modo multiusuario completo por consola)
 - **4**: no se utiliza
 - **5**: X11 (modo multiusuario completo con login gráfico basado en X)
 - **6**: reboot

¿Dónde se define qué Runlevel ejecutar al iniciar el sistema operativo?

Se encuentra definido en `/etc/inittab` → `id:NivelesEjecución:acción:proceso`.

- Los scripts que se ejecutan están en `/etc/init.d`
- En `/etc/rcX.d` (donde `X = 0..6`) hay links a los archivos del `/etc/init.d`
- Formato de los links:
`[S|K]<orden><nombreScript>`

```
$ ls -l /etc/rcS.d/
S55urandom
S70x11-common
```
- **S**: lanza el script con el argument start
- **K**: lanza el script con el argument stop

¿Todas las distribuciones respetan estos estándares?

No todas las distribuciones respetan estrictamente los estándares de runlevel.

E. Archivo `/etc/inittab`. ¿Cuál es su finalidad? ¿Qué tipo de información se almacena en él? ¿Cuál es la estructura de la información que en él se almacena?.

Encontramos la configuración principal de los Runleves, con el siguiente formato :

- `id:nivelesEjecución:acción:proceso`

- F. Suponga que se encuentra en el runlevel . Indique qué comando(s) ejecutaria para cambiar al runlevel . ¿Este cambio es permanente? ¿Por qué?.
- G. Scripts RC. ¿Cuál es su finalidad? ¿Dónde se almacenan? Cuando un sistema GNU/Linux arranca o se detiene se ejecutan scripts, indique cómo determina qué script ejecutar ante cada acción. ¿Existe un orden para llamarlos? Justifique.

4. Usuarios :

- A. ¿Qué archivos son utilizados en un sistema GNU/Linux para guardar la información de los usuarios?.

La información de los usuarios en un Sistema Gnu/Linux se guarda en:

> /etc/passwd : En este archivo podemos encontrar la información de los usuarios . La cual se organiza de la siguiente manera :

Ej : `usuario:x:1001:1001:Usuario de Ejemplo,,,:/home/usuario:/bin/bash`
nombre de usuario,"Contraseña", User ID, Group ID, Información, el directorio home y el shell predeterminado.

> /etc/shadow : En este archivo encontramos las contraseñas de los usuarios. Estas están encriptadas.

Ej : `usuario:$6$saltsalt$hashedpassword:18000:0:99999:7:::`

> /etc/group : En este archivo encontramos la información de los grupos del sistema.

Ej : `grupo:x:1001:usuario1,usuario2`

Nombre del grupo, 'Contraseña', el GID y los usuarios que pertenecen a cada grupo separados por comas.

> /etc/gshadow : Similar al shadow , pero este es para las contraseñas de los grupos.

- B. ¿A qué hacen referencia las siglas UID y GID? ¿Pueden coexistir UIDs iguales en un sistema GNU/Linux? Justifique.

UID (User ID): Es un identificador único asignado a cada usuario en el sistema. Se utiliza para identificar al usuario en las operaciones del sistema, como la gestión de permisos y recursos.

GID (Group ID): Es un identificador único asignado a cada grupo en el sistema. Similar al UID, se utiliza para gestionar permisos y asignar recursos a grupos de usuarios.

Cada UID debe ser único para garantizar que el sistema pueda identificar correctamente a cada usuario y gestionar los permisos y accesos de manera adecuada. Si hubiera usuarios con el mismo UID, el sistema no podría distinguir entre ellos, lo que llevaría a conflictos en el acceso a archivos y recursos, además de problemas de seguridad.

- C. ¿Qué es el usuario root? ¿Puede existir más de un usuario con este perfil en GNU/Linux? ¿Cuál es la UID del root?.

En sistemas operativos del tipo Unix, el superusuario o root es el nombre convencional de la cuenta de usuario que posee control total sobre el sistema.

EL UID del usuario root o superusuario es el UID 0. Como se mencionó en el punto anterior el UID es único por lo cual solo se debe tener un usuario root, pero en la práctica es posible asignarle a cualquier usuario el UID 0 y que este tenga los privilegios del usuario root.

- D. Agregue un nuevo usuario llamado iso2017 a su instalación de GNU/Linux, especifique que su home sea creada en /home/iso_2017, y hágalo miembro del grupo cátedra (si no existe, deberá crearlo). Luego, sin iniciar sesión como este usuario cree un archivo en su home personal que le pertenezca. Luego de todo esto, borre el usuario y verifique que no queden registros de él en los archivos de información de los usuarios y grupos.

```
lauti@ISO2024: ~  
root@ISO2024:/home/lauti# sudo groupadd catedra  
root@ISO2024:/home/lauti# sudo useradd -d "/home/iso_2024" -m iso2024  
root@ISO2024:/home/lauti# sudo usermod -a -G catedra iso2024  
root@ISO2024:/home/lauti# id -nG iso2024  
iso2024 catedra  
root@ISO2024:/home/lauti# touch /home/iso_2024/archivo  
root@ISO2024:/home/lauti# ls /home/iso_2024/  
archivo  
root@ISO2024:/home/lauti# sudo userdel iso2024  
root@ISO2024:/home/lauti# cat etc/passwd | grep -w "iso2024"  
:at: etc/passwd: No such file or directory  
root@ISO2024:/home/lauti# cat /etc/passwd | grep -w "iso2024"  
root@ISO2024:/home/lauti#
```

E. Investigue la funcionalidad y parámetros de los siguientes comandos:

5. FileSystem :

A. ¿Cómo son definidos los permisos sobre archivos en un sistema GNU/Linux?

En un sistema GNU/Linux, los permisos sobre archivos se definen mediante un sistema de control de acceso basado en tres categorías: propietario, grupo y otros. Cada archivo y directorio tiene asociado un conjunto de permisos que determinan quién puede leer, escribir o ejecutar el archivo.

Propietario: El usuario que creó el archivo o directorio.

Grupo: Un conjunto de usuarios que tienen acceso al archivo; todos los miembros de este grupo comparten los mismos permisos.

Otros: Todos los demás usuarios que no son el propietario ni pertenecen al grupo.

Existen tres tipos de permisos :

- > **Lectura (r):** Permite leer el archivo o listar el contenido del directorio.
- > **Escritura (w):** Permite modificar el archivo o agregar/eliminar archivos en el directorio.
- > **Ejecutar (x):** Permite ejecutar el archivo como un programa o acceder al directorio.

Ej : `-rwxr-xr-- 1 usuario grupo 123 oct 4 10:00 mi_script.sh`

- > El primer campo indica el tipo : (-) archivo , (d) directorio, (l) enlace simbólico.
- > Los 9 campos siguientes los permisos : (3)propietario,(3)grupo,(3)otros.
- > nombre de usuario, grupo , fecha, hora y por último el nombre del archivo.

B. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con los permisos en GNU/Linux: `chmod` `chown` `chgrp`.

CHMOD : Funcionalidad: Cambia los permisos de acceso a archivos y directorios.

Parámetros comunes:

- **u**: Propietario (user).
- **g**: Grupo (group).
- **o**: Otros (others).
- **a**: Todos (all).
- **+**: Añadir un permiso.
- **-**: Quitar un permiso.
- **=**: Establecer permisos específicos.

EJ : `chmod u+x archivo` → Le da permiso de ejecución al propietario.

(Hay otra forma utilizando la notación octal , la explico en el próximo ejercicio).

CHOWN : Funcionalidad: Cambia el propietario y/o el grupo de un archivo o directorio.

Parámetros comunes:

- **usuario**: Nombre del nuevo propietario.
- **grupo**: Nombre del nuevo grupo (opcional :nombredegrupo).

EJ : `chown Usuario archivo.sh` → cambia el propietario a archivo.sh

CHGRP : Funcionalidad: Cambia el grupo asociado a un archivo o directorio.

Parámetros comunes:

- **grupo**: Nombre del nuevo grupo.

C. Al utilizar el comando `chmod` generalmente se utiliza una notación octal asociada para definir permisos. ¿Qué significa esto? ¿A qué hace referencia cada valor?

- Existen 3 tipos de permisos y se basan en una notación octal:

Permiso	Valor	Octal
Lectura	R	4
Escritura	W	2
Ejecución	X	1

- Se aplican sobre los usuarios:
 - Usuario: permisos del dueño → **U**
 - Usuario: permisos del grupo → **G**
 - Usuario: permisos de otros usuario → **O**
- Se utiliza el comando **chmod**:

```
$ chmod 755 /tmp/script
```

Se suman los valores únicos de los tipos de permisos y asignamos qué tipo le damos a cada usuario en las tres posiciones ingresadas después del `chmod`

- D. ¿Existe la posibilidad de que algún usuario del sistema pueda acceder a determinado archivo para el cual no posee permisos? Nombrelo, y realice las pruebas correspondientes.

Si un usuario tiene privilegios de **sudo**, puede ejecutar comandos como superusuario (root) y, por lo tanto, puede acceder a cualquier archivo del sistema, independientemente de los permisos.

- E. Explique los conceptos de “full path name” y “relative path name”. De ejemplos claros de cada uno de ellos.

Full Path Name (Ruta Completa)

La ruta completa es la dirección completa que señala a un archivo o directorio desde la raíz del sistema de archivos. Siempre comienza con una barra diagonal /, que representa la raíz del sistema recopilando todos los directorios por los que pasa hasta llegar a donde está ahora.

EJ : **/home/lauti/facultad/iso/practica2/resolucionPractica2.doc**

Relative Path Name (Ruta Relativa)

La ruta relativa es la dirección de un archivo o directorio en relación a la ubicación actual en el sistema de archivos. No comienza con una barra diagonal / y depende del directorio en el que estés en ese momento.

EJ. Teniendo en cuenta el ejemplo anterior la ruta relativa :

> practica2/resolucionPractica2.doc

- F. ¿Con qué comando puede determinar en qué directorio se encuentra actualmente?
¿Existe alguna forma de ingresar a su directorio personal sin necesidad de escribir todo el path completo? ¿Podría utilizar la misma idea para acceder a otros directorios? ¿Cómo? Explique con un ejemplo.

> Con el comando **PWD** nos dirá donde estamos actualmente mostrándonos la ruta absoluta.

> Para ingresar al directorio personal sin la necesidad de ingresar todo el path completo lo hacemos usando el comando **CD** solo sin parámetros.

> Utilizando : **CD ~USUARIO** accedemos al home de otro usuario.

- G. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con el uso del FileSystem:

> **cd** : Cambia el directorio de trabajo actual en la línea de comandos.

> **unmount** : Desmonta un sistema de archivos.

> **mkdir** : Crea uno o más directorios.

> **du** : Muestra el uso del espacio en disco de archivos y directorios.

> **rmdir** : Elimina directorios vacíos.

> **df** : Muestra el uso del espacio en disco de los sistemas de archivos montados.

> **mount** : Monta un sistema de archivos en un punto de montaje.

> **ln** : Crea enlaces a archivos, que pueden ser enlaces duros o simbólicos.

> **ls** : Lista el contenido de un directorio.

Parámetros:

- **-l**: Muestra una lista detallada, incluyendo permisos, propietario, tamaño y fecha de modificación.
- **-h**: Muestra tamaños en un formato legible (K, M, G).
- **-R**: Lista recursivamente el contenido de los subdirectorios.

> pwd : Muestra el directorio de trabajo actual (Print Working Directory).

> cp : Copia archivos y directorios.

Parámetros:

- **-r**: Copia recursivamente directorios y su contenido.

> mv : Mueve o renombra archivos y directorios.

Parámetros:

- **-i**: Pide confirmación antes de sobrescribir archivos.
- **-v**: Muestra detalles del movimiento o renombrado.
- **-n**: No sobrescribe archivos existentes.

6. Procesos :

A. ¿Qué es un proceso? ¿A que hacen referencia las siglas PID y PPID? ¿Todos los procesos tienen estos atributos en GNU/Linux?.

Un proceso es un programa que se está ejecutando. (usamos como sinónimos tarea, job y proceso)

PID (Process ID) : Es el identificador del proceso.

PPID (Parent Process ID) : Es el identificador de un proceso padre. En linux todos los procesos tienen un PPID menos el proceso init o systemd ya que son el primer proceso en ejecutarse.

Tienen otros atributos como Estado, espacio en memoria, prioridad de planificación, etc.

B. Indique qué comandos se podrían utilizar para ver qué procesos están en ejecución en un sistema GNU/Linux.

> Ps : Lista todos los procesos que estén corriendo en el sistema.

> Top : Nos da un informe en tiempo real de los procesos.

C. ¿Qué significa que un proceso se está ejecutando en Background? ¿Y en Foreground?

En GNU/Linux, los procesos pueden ejecutarse en **background** (en segundo plano) o **foreground** (en primer plano).

Proceso en Foreground (en primer plano) : Mientras que un proceso se está ejecutando en primer plano, no se admite el uso de la consola hasta que este termine el proceso.

Proceso en Background (en segundo plano) : Mientras se ejecuta el proceso en segundo plano la shell se libera y puede ser usada mientras este se termina de ejecutar.

D. ¿Cómo puedo hacer para ejecutar un proceso en Background? ¿Como puedo hacer para pasar un proceso de background a foreground y viceversa?

Para ejecutar un proceso en background en Linux, se puede agregar el símbolo "&" al final del comando en la línea de comandos.

Ej : `cat archivo.doc &`

Para pasar de un proceso a otro debemos ejecutar el comando (bg) → de primer a segundo plano y (fg) → de segundo a primer plano. Si se esta ejecutando cortamos la ejecución con (Ctrl + z).

E. Pipe (|). ¿Cuál es su finalidad? Cite ejemplos de su utilización.

El comando pipe conecta la salida estándar del comando con la entrada estándar del siguiente comando.

EJ : Si quiero listar todos los nombre de usuarios.

```
cat etc/passwd | cut -d: -f1
```

F. Redirección. ¿Qué tipo de redirecciones existen? ¿Cuál es su finalidad? Cite ejemplos de utilización.

Las redirecciones consisten en trasladar información de un tipo a otro

- Al utilizar redirecciones mediante `>` (destruktiva):
 - Si el archivo de destino no existe, se lo crea
 - Si el archivo existe, se lo trunca y se escribe el nuevo contenido
- Al utilizar redirecciones mediante `>>` (no destructiva):
 - Si el archivo de destino no existe, se lo crea
 - Si el archivo existe, se agrega la información al final

G. Comando kill. ¿Cuál es su funcionalidad? Cite ejemplos.

Se utiliza para enviar señales a procesos, principalmente para terminar procesos en ejecución. (/bin/kill)

H. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con el manejo de procesos en GNU/Linux. Además, compárelos entre ellos:

> ps : Muestra información sobre los procesos en ejecución.

Parámetros:

- `-e` o `-A`: Muestra todos los procesos.
- `-f`: Muestra información en formato completo.
- `-u user`: Muestra solo los procesos de un usuario específico.
- `-aux`: Muestra todos los procesos con información detallada (es un comando comúnmente usado).

> kill : Envía señales a procesos, generalmente para terminarlos.

Parámetros:

- `-9`: Fuerza la terminación del proceso (SIGKILL).
- `-15`: Envía la señal de terminación (SIGTERM) de manera más amable (es la señal predeterminada).
- `pid`: El ID del proceso al que se le enviará la señal.

> pstree : Muestra los procesos en forma de árbol, mostrando la jerarquía entre ellos.

> killall : Envía señales a todos los procesos que coinciden con el nombre dado.

Parámetros:

- `-9`: Fuerza la terminación de los procesos.
- `-15`: Envía la señal de terminación (SIGTERM) de forma amable.
- `name`: El nombre del proceso al que se le enviará la señal (ej. `firefox`).

> `top` : Muestra una lista dinámica de los procesos en ejecución y su uso de recursos (CPU, memoria, etc.).

`-u user`: Muestra solo los procesos de un usuario específico.

> `nice` : Cambia la prioridad de ejecución de un proceso. Un valor más bajo significa mayor prioridad.

Parámetros:

- `-n value`: Establece el valor de "nice" (prioridad) del proceso, donde el valor predeterminado es 0.
- `command`: El comando a ejecutar con la prioridad modificada (ej. `nice -n 10 command`).

7. Otros comandos de Linux.

A. ¿A qué hace referencia el concepto de empaquetar archivos en GNU/Linux?

El concepto de empaquetar archivos en GNU/Linux se refiere al proceso de combinar varios archivos y/o directorios en un solo archivo. Esto se hace principalmente por motivos de organización, compresión y facilidad de distribución.

B. Seleccione 4 archivos dentro de algún directorio al que tenga permiso y sume el tamaño de cada uno de estos archivos. Cree un archivo empaquetado conteniendo estos 4 archivos y compare los tamaños de los mismos. ¿Qué característica nota?

C. ¿Qué acciones debe llevar a cabo para comprimir 4 archivos en uno solo? Indique la secuencia de comandos ejecutados.

Comprimir un archivo es el proceso de reducir su tamaño para facilitar el almacenamiento y la transferencia. Esto se logra mediante algoritmos de compresión que eliminan redundancias y optimizan la representación de los datos.

Para comprimir y descomprimir archivos utilizamos :

> `gzip` (Archivo)

> **gunzip** (Archivo)

También podemos empaquetar y comprimir archivos a través de una sola instrucción con :
tar -czvf

Comando	Explicación
<pre>tar -cvf archivos.tar archivo1.txt archivo2.txt archivo3.txt archivo4.txt</pre>	Empaquetar: tar crea un archivo empaquetado (archivos.tar) que contiene los archivos listados. - -c : Crea un nuevo archivo tar. - -v : Muestra el progreso (modo verbose). - -f : Especifica el nombre del archivo tar.
<pre>tar -czvf archivos.tar.gz archivo1.txt archivo2.txt archivo3.txt archivo4.txt</pre>	Empaquetar y Comprimir: Crea un archivo tar comprimido (archivos.tar.gz). - -z : Comprime el archivo tar usando gzip. - -c : Crea un nuevo archivo tar. - -v : Muestra el progreso. - -f : Especifica el nombre del archivo tar.

El archivo resultante tiene como terminación **.tar** ya que indica que fue empaquetado con este comando y después puede tener un **.gz** que es comprimido con gzip (Zip en windows).

D. ¿Pueden comprimirse un conjunto de archivos utilizando un único comando?

Se pueden empaquetar y comprimir con el ejemplo del punto anterior usando **tar -czvf**, pasando el nombre del nuevo archivo y listando los que se quieren empaquetar y comprimir en este.

E. Investigue la funcionalidad de los siguientes comandos

> **tar** : Utilizado para archivar múltiples archivos y directorios en un solo archivo (generalmente con la extensión **.tar**).

Parámetros:

- **-c**: Crea un nuevo archivo tar.
- **-x**: Extrae archivos de un archivo tar.
- **-f**: Especifica el nombre del archivo tar.
- **-v**: Muestra el progreso de los archivos que se están procesando (verbose).
- **-z**: Comprime el archivo tar con gzip (generalmente se utiliza como **.tar.gz**).

> grep : Busca líneas que coinciden con un patrón en uno o más archivos.

Parámetros:

- -i: Ignora diferencias entre mayúsculas y minúsculas.
- -r: Busca recursivamente en directorios.
- -v: Muestra líneas que **no** coinciden con el patrón.
- -n: Muestra el número de línea de cada coincidencia.
- -E: Permite usar expresiones regulares extendidas.
- -w : se utiliza para buscar coincidencias de palabras completas.

> gzip : Comprime archivos para reducir su tamaño, generando archivos con la extensión .gz .

> zgrep : Busca texto dentro de archivos comprimidos con gzip (.gz), usando **grep** internamente.

> wc : Cuenta líneas, palabras y caracteres en archivos de texto.

Parámetros:

- -l: Cuenta solo las líneas.
- -w: Cuenta solo las palabras.
- -c: Cuenta solo los caracteres.
- -m: Cuenta solo los caracteres (incluso en UTF-8).
- -L: Muestra la longitud de la línea más larga.

8. Ejercicio de comandos :

Indique qué acción realiza cada uno de los comandos indicados a continuación considerando su orden. Suponga que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asuma que se encuentra posicionado en el directorio de trabajo del usuario con el que se loguea). En caso de no poder ejecutarse el comando, indique la razón:

```
ls -l > prueba
ps > PRUEBA
chmod 710 prueba
chown root : root PRUEBA
chmod 777 PRUEBA
chmod 700 /etc/passwd
```

```
passwd root
rm PRUEBA
man /etc/shadow
find / -name *.conf
usermod root -d /home/newroot -L
cd /root
rm *
cd /etc
cp * /home -R
shutdown
```

- ls -l > prueba : Lo que hace es listar todos los archivos de manera detallada y los guarda en el archivo prueba (En caso de no existir lo crea / agrega al final).
- ps > PRUEBA : Lista toda la información sobre los procesos en ejecución del sistema y los manda al archivo creado (En caso de no existir) PRUEBA.
- chmod 710 prueba : Da permisos a los usuarios para el archivo prueba , (dueño) le da 7 por ende da permiso de rwx , (grupo) de x , (Otros) ningún permiso.
- chown root : root PRUEBA : Lo que hace es intentar cambiar el (propietario y grupo) al archivo PRUEBA (root,root). **No puede hacerlo.**
- chmod 777 PRUEBA : Da todos los permisos rwx a todos los usuarios.
- chmod 700 /etc/passwd : Intenta cambiar todos los archivos de /etc/passwd . **NO se puede por querer hacerlo en esta carpeta importante.**
- passwd root : Intenta cambiar la contraseña de un usuario (ROOT) por la que sea ingresada. **No es posible ya que solo lo puede hacer el root.**
- rm PRUEBA : Se elimina el archivo prueba.
- man /etc/shadow : Intenta recibir un manual de shadow. **No recibe rutas el comando man.**
- find / -name *.conf : Busca y lista desde (Raiz /) archivos que su nombre termine con ".conf".
- usermod root -d /home/newroot -L : Intenta modificar el home del usuario (root) y desactivar la contraseña. **Pero si el usuario no es root no deja ejecutar ese comando.**
- cd /root : Se posiciona en la carpeta root pero **falla ya que no posee permisos.**
- rm * : Borra todos los archivos del directorio en donde estamos (Solo archivos).
- cd /etc : Se posiciona en la carpeta /etc.
- cp * /home -R : Intenta copiar de manera recursiva todos los archivos de /etc en home. **No posee los permisos para hacerlo.**
- shutdown : Apaga el SO.

9. Indicar que comandos ejecutan la acción :

> A. Terminar el proceso con PID 23 : **kill -9 23** (Le paso el -9 para la terminación y el PID).

> B. Terminar el proceso llamado init o systemd. ¿Qué resultados obtuvo?: `kill -9 1` (No se puede matar al proceso init) .

> C : Buscar todos los archivos de usuarios en los que su nombre contiene la cadena ".conf": `find / -type f -name "*.conf"` .

> D. Guardar una lista de procesos en ejecución al archivo /home//procesos :
`ps > /home/lauti/procesos`.

> E. Cambiar los permisos del archivo /home//xxxx a: Usuario: Lectura, escritura, ejecución
Grupo: Lectura, ejecución Otros: ejecución .

`chmod 751 /home/lauti/xxxx`

> F. Cambiar los permisos del archivo /home//yyyy a: Usuario: Lectura, escritura. Grupo:
Lectura, ejecución Otros: Ninguno.

`chmod 650 /home/lauti/yyyy`.

> G. Borrar todos los archivos del directorio /tmp: `rm /tmp/*` (Borra solo los archivos).

> H. Cambiar el propietario del archivo /opt/isodata al usuario iso2010 :
`chown iso2010 /opt/isodata`

> I. Guardar en el archivo /home//donde el directorio donde me encuentro en este momento,
en caso de que el archivo exista no se debe eliminar su contenido anterior :

`pwd >> /home/lauti/donde` (>> No se es destructivo).

10. Indicar que comandos ejecutan la acción :

A . Ingrese al sistema como usuario "root":

`su`

{ingresamos la contraseña que nos pide}.

B. Cree un usuario. Elija como nombre, por convención, la primer letra de su nombre
seguida de su apellido. Asígnele una contraseña de acceso:

`adduser -m lbudini`

`passwd lbudini {ingresamos la contraseña}`

C. ¿Qué archivos fueron modificados luego de crear el usuario y qué directorios se crearon?
Fueron modificados los de /etc/passwd , /group , /shadow y se creo el directorio
/home/lbudini.

D. Crear un directorio en /tmp llamado cursada2017 : `mkdir /tmp/cursada2017`

E. Copiar todos los archivos de /var/log al directorio antes creado:

```
cp /var/log /temp/cursada2017
```

F. Para el directorio antes creado (y los archivos y subdirectorios contenidos en él) cambiar el propietario y grupo al usuario creado y grupo users:

```
chown -R lbudini:users /temp/cursada2017
```

G. Agregue permiso total al dueño, de escritura al grupo y escritura y ejecución a todos los demás usuarios para todos los archivos dentro de un directorio en forma recursiva:

```
chmod -R 723 /tmp/cursada2017
```

H. Acceda a otra terminal virtual para loguearse con el usuario antes creado:

```
su - lbudini
```

I. Una vez logueado con el usuario antes creado, averigüe cuál es el nombre de su terminal:

```
cat etc/passwd | grep -w "lbudini" | cut -d: -f7
```

J. Verifique la cantidad de procesos activos que hay en el sistema:

```
ps -e | wc -l
```

K . Verifiqué la cantidad de usuarios conectados al sistema :

```
who | wc -l      tamb → users | wc -l
```

L . Vuelva a la terminal del usuario root, y envíele un mensaje al usuario anteriormente creado, avisándole que el sistema va a ser apagado :

```
write lbudini
```

Mensajes

```
ctrl +D
```

M . Apague el sistema: `shutdown now`

11. Indicar que comandos ejecutan la acción :

(a) Cree un directorio cuyo nombre sea su número de legajo e ingrese a él :

```
mkdir 21290 → Lo creo
```

```
cd 21290 → Ingreso en el directorio.
```

(b) Cree un archivo utilizando el editor de textos vi, e introduzca su información personal: Nombre, Apellido, Número de alumno y dirección de correo electrónico. El archivo debe llamarse "LEAME":

```
vi LEAME → (i) para insertar cod , luego (Esc) , luego (:wq) para guardar.
```

(c) Cambie los permisos del archivo LEAME, de manera que se puedan ver reflejados los siguientes permisos: Dueño: ningún permiso Grupo: permiso de ejecución Otros: todos los permisos:

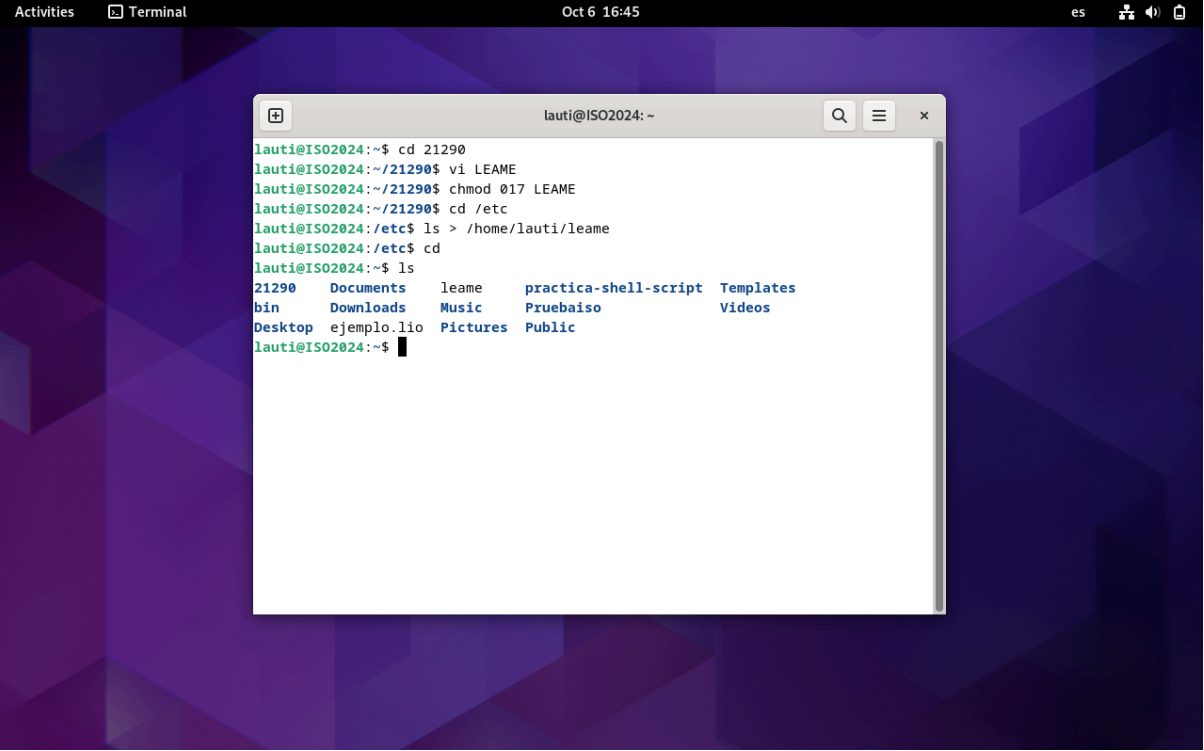
```
chmod 017 LEAME
```

(d) Vaya al directorio /etc y verifique su contenido. Cree un archivo dentro de su directorio personal cuyo nombre sea leame donde el contenido del mismo sea el listado de todos los archivos y directorios contenidos en /etc. ¿Cuál es la razón por la cuál puede crear este archivo si ya existe un archivo llamado "LEAME" en este directorio?:

```
cd /etc
```

```
verifique (?)
```

```
ls > /home/lauti/leame (Ya que linux al ser case sensitive no tendrían el mismo nombre).
```



```
Activities Terminal Oct 6 16:45 es
lauti@ISO2024: ~
lauti@ISO2024:~$ cd 21290
lauti@ISO2024:~/21290$ vi LEAME
lauti@ISO2024:~/21290$ chmod 017 LEAME
lauti@ISO2024:~/21290$ cd /etc
lauti@ISO2024:/etc$ ls > /home/lauti/leame
lauti@ISO2024:/etc$ cd
lauti@ISO2024:~$ ls
21290  Documents  leame  practica-shell-script  Templates
bin    Downloads  Music  Pruebasiso             Videos
Desktop ejemplo.lio Pictures Public
```

(e) ¿Qué comando utilizaría y de qué manera si tuviera que localizar un archivo dentro del filesystem? ¿Y si tuviera que localizar varios archivos con características similares? Explique el concepto teórico y ejemplifique.

```
find / -name "nombre o extensión" → * comodín para autocompletar.
```

```
find / -iname "nombre o extensión"
```

(f) Utilizando los conceptos aprendidos en el punto e), busque todos los archivos cuya extensión sea .so y almacene el resultado de esta búsqueda en un archivo dentro del directorio creado en a). El archivo deberá llamarse .ejercicio_f".

```
find / -name "*.so" > /home/lauti/21290/.ejercicio_f
```

12. Comandos :

Indique qué acción realiza cada uno de los comandos indicados a continuación considerando su orden. Suponga que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asuma que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando indique la razón:

`mkdir iso` → Crea un directorio llamado iso

`cd ./iso ; ps > f0` → Se posiciona en iso y manda los procesos en ejecución q muestra el comando ps al archivo (si no existe lo crea) f0.

`ls > f1` → Guarda en el archivo f1 (Si no existe lo crea) la salida del comando ls.

`cd /` → Se posiciona en raíz (/)

`echo $HOME` → Imprime en pantalla el directorio que se esta usando.

`ls -l $> $HOME/iso/ls` →

`cd $HOME; mkdir f2`

`ls -ld f2`

`chmod 341 f2`

`touch dir`

`cd f2`

`cd ~/iso`

`pwd >f3`

`ps | grep 'ps' | wc -l >> ../f2/f3`

`chmod 700 ../ f2 ; cd ..`

`find . -name etc/passwd`

`find / -name etc/passwd`

`mkdir ejercicio5`

TERMINAR

14. Indicar que comandos ejecutan la acción :

(a) Cree un directorio llamado logs en el directorio /tmp.

`mkdir /tmp/logs`

(b) Copie todo el contenido del directorio /var/log en el directorio creado en el punto anterior.

```
cp -r /var/log/* /tmp/logs
```

(c) Empaquete el directorio creado en 1, el archivo resultante se debe llamar "misLogs.tar".

```
tar -cvf misLogs.tar /tmp/logs
```

(d) Empaquete y comprima el directorio creado en 1, el archivo resultante se debe llamar "misLogs.tar.gz".

```
tar -czvf misLogs.tar.gz /tmp/logs
```

(e) Copie los archivos creados en 3 y 4 al directorio de trabajo de su usuario.

```
cp miLogs.tar /home/lauti
```

```
cp miLogs.tar.gz /home/lauti
```

(f) Elimine el directorio creado en 1, logs.

```
rm -r /tmp/logs
```

(g) Desempaquete los archivos creados en 3 y 4 en 2 directorios diferentes.

```
mkdir /home/lauti/primero /home/lauti/segundo
```

```
tar -xvf misLogs.tar -C /home/lauti/primero
```

```
tar -xzvf misLogs.tar.gz -C /home/lauti/segundo
```

{Con -C cambiamos la ruta en donde se desempaqueta}